parameter name. For example, to set the `id` route parameter, you'd set the `asp-route-id` value. If the `ProductId` property of your `PageModel` contains the `id` value you require, you could use

```
<form asp-page="Product" asp-route-id="@Model.ProductId">
```

Based on the route template of the Product.cshtml Razor Page (and assuming `ProductId=5` in this example), this would generate the following markup:

```
<form action="/Product/5" method="post">
```

You can add as many `asp-route-*` attributes as necessary to your `<form>` to generate the correct `action` URL. You can also set the Razor Page handler to use the `asp-page-handler` attribute. This ensures that the form `POST` will be handled by the handler you specify.

NOTE The Form Tag Helper has many additional attributes, such as `asp-action` and `asp-controller`, that you generally won't use with Razor Pages. Those are useful only if you're using MVC controllers with views. In particular, look out for the `asp-route` attribute—this is *not* the same as the `asp-route-*` attribute. The former is used to specify a named route (such as a named minimal API endpoint), and the latter is used to specify the route *values* to use during URL generation.

The main job of the Form Tag Helper is to generate the `action` attribute, but it performs one additional important function: generating a hidden `<input>` field needed to prevent *cross-site request forgery* (CSRF) attacks.

DEFINITION *Cross-site request forgery* (CSRF) attacks are a website exploit that can allow actions to be executed on your website by an unrelated malicious website. You'll learn about them in detail in chapter 29.